ARCHITECTURE & TECHNICAL REPORT

Task Analyzer Agent

AI-powered webhook agent that analyzes project management tasks and generates development plans using Claude AI

VERSION	DATE	RUNTIME	AI MODEL
1.0 — Phase 1	March 2026	Node.js + TypeScript	Claude Sonnet 4.5

INTEGRATIONS

GitHub · Trello · Asana

Table of Contents

1. System Overview
2. Architecture Diagram
3. Component Reference
4. Request Flow
5. Challenges & Resolutions
6. Security: Fixed & Remaining
7. Future Improvements
8. Environment Variables

1 System Overview

The **Task Analyzer Agent** is a Node.js/TypeScript webhook service that listens for events from project management tools (GitHub Issues, Trello, Asana) and uses **Claude AI** to automatically analyze the incoming task. Based on the analysis, it either:

📋

Writes a Development Plan

When a task is sufficiently detailed, the agent produces a structured plan including technical approach, files to change, test plan, risks, and effort estimate.

❓

Asks Clarifying Questions

When a task lacks necessary detail (too short, vague goal, missing fields), the agent posts targeted questions back to the task as a comment.

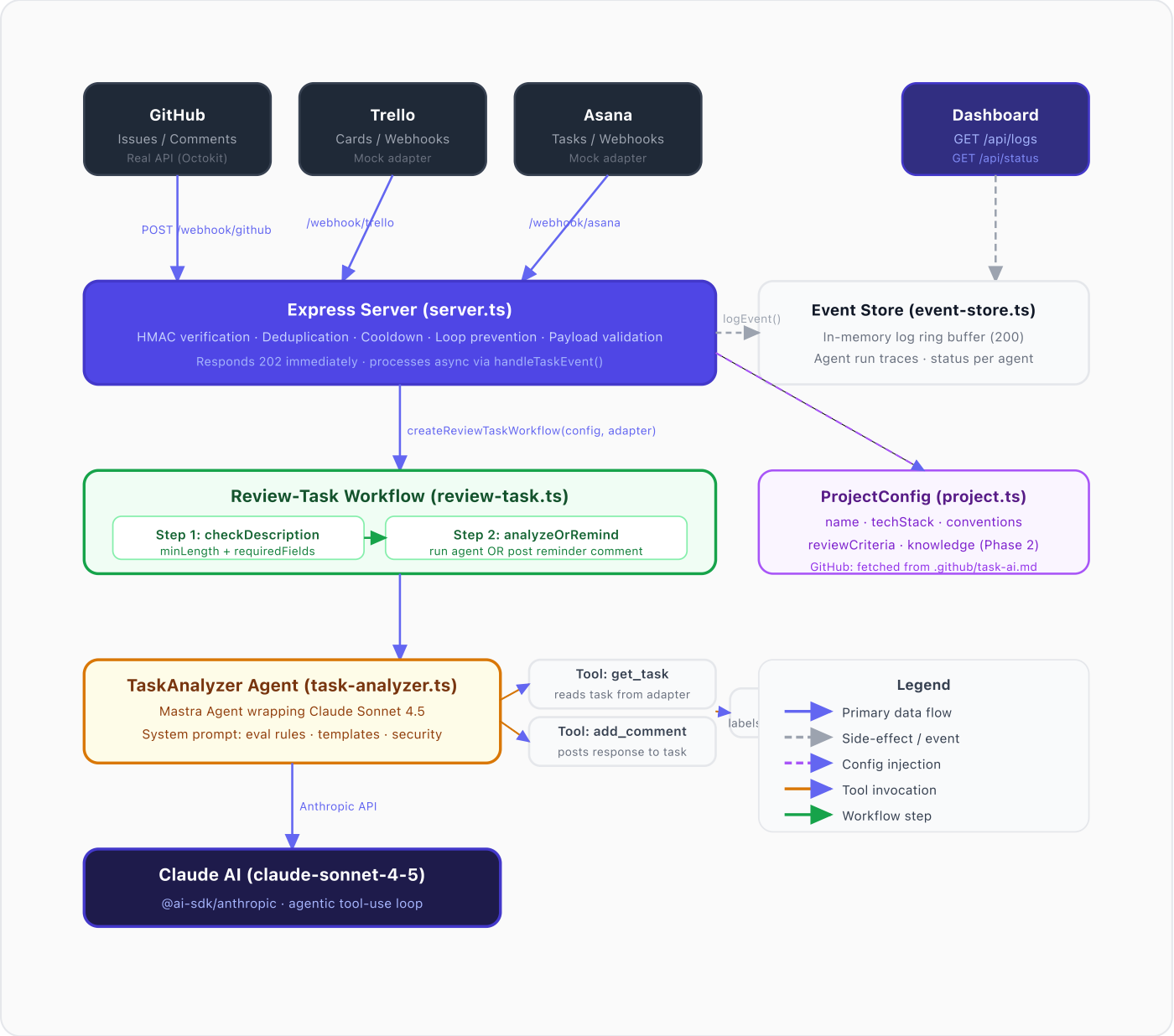
The system also supports a **conversation mode** — when users reply to clarifying questions via a mention comment (e.g., `@task-ai`), the agent re-reads the full thread, incorporates the new answers, and either produces a plan or asks remaining questions.

CURRENT STATUS

Phase 1 is complete: GitHub Issues integration is fully functional with real API calls. Trello and Asana adapters are implemented as mocks pending real API credentials (Phase 2).

2 Architecture Diagram

The diagram below shows all major system components and how they interact at runtime.



3 Component Reference

Project File Structure

```
src/
├── mastra/
│ ├── agents/
│ │ └── task-analyzer.ts ← createTaskAnalyzerAgent(config, adapter) factory
│ ├── tools/
│ │ ├── index.ts ← createTools(adapter) — 3 tools
│ │ └── rag.ts ← Phase 2 stubs (searchDocs, searchCode)
│ └── workflows/
│ └── review-task.ts ← createReviewTaskWorkflow(config, adapter)
├── adapters/
│ ├── interface.ts ← ProjectManagerAdapter, Task, TriggerContext
│ ├── github.ts ← GitHubAdapter (real Octokit API)
│ ├── trello.ts ← TrelloAdapter (mock)
│ └── asana.ts ← AsanaAdapter (mock)
├── config/
│ └── project.ts ← ProjectConfig interface + defaultProjectConfig
├── store/
│ └── event-store.ts ← In-memory log + agent run trace store
├── server.ts ← Express server + all webhook handlers
└── index.ts ← Entry point, env validation, port check
```

API Endpoints

Method	Path	Description	Auth
`POST`	`/webhook/github`	GitHub Issues / Comments webhook receiver. Validates HMAC, deduplicates, enforces cooldown.	HMAC signed
`POST`	`/webhook/trello`	Trello card event receiver (mock adapter).	Unverified
`POST`	`/webhook/asana`	Asana task event receiver (mock adapter).	Unverified
`GET`	`/health`	Returns `{ status: "ok", adapters: [...] }`. Shows which adapters are active.	Public
`GET`	`/api/status`	Returns in-memory agent status map (last task, last run, recent traces).	No auth
`GET`	`/api/logs`	Returns last 100 log events from in-memory ring buffer.	No auth
`GET`	`/`	Serves static dashboard from `public/index.html`.	Public

Adapter Interface

All PM integrations implement a single `ProjectManagerAdapter` contract, enabling the workflow and agent to be fully platform-agnostic:

Method	Signature	Description
`getTask`	`(taskId) → Promise<Task>`	Fetch task details by ID
`addComment`	`(taskId, text) → Promise<void>`	Post analysis result as a comment
`setStatus`	`(taskId, status) → Promise<void>`	Update task status / labels

The `GitHubAdapter` also adds `getComments()` and `fetchRepoConfig()` — GitHub-specific extensions used for conversation mode and per-repo configuration.

Agent Tools

The Mastra agent is given exactly three tools, each delegating directly to the active adapter:

Tool ID	Purpose	Input
`get_task`	Retrieve full task data before analysis	`taskId: string`
`add_comment`	Post the plan or questions back to the task	`taskId, comment: string`
`set_status`	Label the task as `ready_for_dev` or `needs_clarification`	`taskId, status: enum`

4 Request Flow

GitHub Issues — Label Trigger

This is the primary, fully-implemented flow:

#	Step	Actor	Details
1	Label applied	GitHub	User adds the `ai-review` label to an issue
2	Webhook fired	GitHub → Server	`POST /webhook/github` with `x-hub-signature-256` header
3	HMAC verify	Server	Signature checked with `crypto.timingSafeEqual`
4	Dedup check	Server	`x-github-delivery` ID checked against seen-set (10-min TTL)
5	Cooldown check	Server	Same issue not re-processed within 60 seconds
6	202 response	Server → GitHub	Immediately returns accepted; processing continues async
7	Fetch config	GitHubAdapter	Reads `.github/task-ai.md` from the repo (fallback: default config)
8	checkDescription	Workflow Step 1	Validates `minDescriptionLength` and `requiredFields`
9a	Fail → reminder	Workflow Step 2	Posts reminder comment, sets `needs-clarification` label
9b	Pass → agent	Workflow Step 2	Runs `TaskAnalyzer` agent with structured prompt
10	Agent loop	Claude AI	Calls `get_task` → evaluates → calls `add_comment` + `set_status`
11	Result posted	GitHubAdapter	Development Plan or Clarifying Questions appear on the issue

GitHub Issues — Comment Trigger (Conversation Mode)

When a user replies to the agent's questions by mentioning `@task-ai` in a comment:

CONVERSATION MODE

The agent loads the full comment thread (up to 50 comments) via `getComments()` and builds a structured prompt containing the task description, the full thread (oldest-first), and the triggering comment. The agent then re-evaluates with all context combined — previous answers count as part of the task spec.

Per-Repo Configuration

For GitHub, each repository can customize agent behavior by creating a `.github/task-ai.md` file with YAML frontmatter:

```yaml
---
name: "My SaaS App"
techStack:
  - "Next.js 14"
  - "PostgreSQL"
conventions:
  - "Feature-based folder structure"
reviewCriteria:
  minDescriptionLength: 100
  requiredFields: ["title", "description", "assignee"]
---
```

If the file is absent, the agent falls back to `defaultProjectConfig` (Next.js + TypeScript + PostgreSQL stack).

5 Challenges & Resolutions

1 Raw Body Capture for HMAC Verification

CHALLENGE

GitHub's webhook signature is computed over the raw request body bytes. Express's `express.json()` middleware parses JSON and discards the raw body by default — making it impossible to verify the HMAC after parsing.

RESOLUTION

Used Express's built-in `verify` callback in `express.json({ verify: (req, res, buf) => { req.rawBody = buf; } })` to capture the raw buffer before it's consumed. The HMAC verification function reads `req.rawBody` directly.

2 Infinite Loop via Bot Self-Comments

CHALLENGE

The agent posts comments on GitHub issues. Those new comments fire `issue_comment` webhook events back to the server. Without protection, the agent would endlessly reply to its own comments, causing an infinite loop and exhausting API quotas.

RESOLUTION

Added a self-comment filter: if `GITHUB_BOT_USERNAME` is set and the comment sender matches, the webhook is silently ignored (`status: "ignored"`). The startup validation also warns operators if `GITHUB_BOT_USERNAME` is missing.

3 GitHub Webhook Delivery Retries

CHALLENGE

GitHub automatically retries webhook deliveries if the server doesn't respond within ~10 seconds (or responds with 5xx). This could cause the same issue to be analyzed multiple times, posting duplicate plans and burning LLM credits.

RESOLUTION

Implemented delivery ID deduplication using a `Map<deliveryId, timestamp>` with a 10-minute TTL. Each delivery's `x-github-delivery` header is checked before processing; duplicates return `200 { status: "duplicate" }` immediately. Expired entries are purged lazily on each webhook call.

4 Rapid Re-triggering via Labels

CHALLENGE

A user could remove and re-add the `ai-review` label in quick succession (or mention the agent multiple times), causing multiple concurrent analyses of the same issue.

RESOLUTION

Added a per-issue cooldown using `issueCooldowns.get(taskId)`: the same issue is not re-processed within 60 seconds of the last accepted trigger. Returns `200 { status: "too_soon" }` to reject rapid

duplicates while avoiding error codes that would trigger retries.

5 Mastra Workflow API Incompatibility

CHALLENGE

The Mastra framework provides a typed `Workflow` builder class for defining multi-step workflows. However, passing adapter and config as closure state between two steps in the builder API required complex generics that created more noise than value for a two-step flow.

RESOLUTION

Implemented the workflow as a plain factory function returning an object with a `.run()` method. This retains the semantic two-step structure (checkDescription → analyzeOrRemind) with clean closure access to `adapter` and `config`, without fighting the framework's type system. A TODO notes migration to the Mastra builder when context-passing improves.

6 Port Conflict on Dual IPv4/IPv6 Stack

CHALLENGE

On modern macOS/Linux systems, `localhost` resolves to `::1` (IPv6). Express can bind on IPv4 while another process owns the IPv6 socket — the two wouldn't conflict at the socket level, but browsers would hit the wrong process.

RESOLUTION

Added an active pre-flight port check using `net.createConnection({ port, host: "localhost" })` before starting the server. If a connection succeeds, the process logs a clear error and exits with code 1 rather than silently starting behind another process.

7 Malformed Payload Crashes

CHALLENGE

Early GitHub webhook handling assumed `payload.issue.number` and `payload.repository.owner.login` would always be present. A missing field caused an uncaught TypeError that crashed the async handler, leaving no response to GitHub and triggering retries.

RESOLUTION

Added explicit null-guards before constructing the `taskId`: `if (!payload?.issue?.number || !payload?.repository?.owner?.login || !payload?.repository?.name)`. Invalid payloads now return `400 { error: "Malformed payload" }` cleanly. Inner error handling also wraps the async processing path with a fallback comment on the issue.

6 Security: Fixed & Remaining

Fixed / Implemented

🔐

GitHub HMAC Signature Verification

All incoming GitHub webhooks are verified using `sha256` HMAC with `crypto.timingSafeEqual()` — preventing timing attacks. Requests without a valid signature return 401.

🔄

Delivery Deduplication

Replay protection via delivery ID tracking (10-min TTL). Prevents the same webhook from being processed twice even if GitHub retries.

⏱️

Per-Issue Rate Cooldown

60-second cooldown per task ID prevents resource exhaustion from rapid label toggle or @mention spam on the same issue.

🔁

Infinite Loop Prevention

Self-comment filter using `GITHUB_BOT_USERNAME` prevents the agent from responding to its own messages and entering an infinite processing loop.

🛡️

Prompt Injection Defense

The agent's system prompt contains explicit "hard rules" that override all user input: attempts to extract instructions, change persona, or inject off-topic commands receive a fixed canned response with no elaboration.

✅

Startup Secret Enforcement

If `GITHUB_TOKEN` is configured but `GITHUB_WEBHOOK_SECRET` is absent, the server refuses to start. This prevents accidentally running without signature verification.

🧪

Payload Null-Guards

All required fields in webhook payloads are validated before use. Missing fields return structured 400 errors rather than crashing the process.

⏳

Agent Timeout

The `agent.generate()` call is wrapped in a 120-second timeout via `withTimeout()`. Stalled LLM calls are killed cleanly rather than holding open connections indefinitely.

Remaining / Open Issues

⚠️

No Webhook Verification for Trello & Asana

Both `/webhook/trello` and `/webhook/asana` accept any POST without signature verification. Any

⚠️

Unauthenticated Internal APIs

`GET /api/status` and `GET /api/logs` expose agent run traces, task IDs, and LLM step details to

actor with the URL can trigger analysis. Marked as TODO in code.

anyone with network access. Should be protected before public deployment.

⚠️

No Global Rate Limiting Middleware

There is no Express rate-limiting middleware (e.g., `express-rate-limit`). All endpoints are susceptible to flood attacks from any source that can reach the server.

⚠️

In-Memory State Not Persistent

The deduplication map, cooldown map, event log, and agent run traces are all in-memory. A server restart loses all deduplication state, potentially allowing duplicate processing of recent events.

⚠️

Thread Truncation Risk

GitHub comment threads are fetched with a hard cap of 50 comments (`per_page: MAX_THREAD_COMMENTS`). For long-running issues, the agent may miss early context. Acknowledged in a code TODO.

⚠️

No Content Security Policy / Helmet

The Express server does not use `helmet.js` or set any security headers (CSP, HSTS, X-Frame-Options). Required before any browser-facing deployment.

7 Future Improvements

Phase 2 — Planned

🔗 **Real Trello & Asana API Integration**

Replace mock adapters with real REST API calls using `TRELLO_API_KEY` / `TRELLO_TOKEN` and `ASANA_TOKEN`. Adapter interface is already defined — only the implementation needs to be added.

🧠 **RAG / Knowledge Base Integration**

The `rag.ts` file already defines stubs for `searchDocsTool` and `searchCodeTool`. When `knowledge.enabled = true` in the project config, the agent should consult the codebase and documentation before writing a plan, producing far more accurate technical approaches.

🔐 **Webhook Signature Verification for Trello & Asana**

Implement HMAC or OAuth verification for Trello and Asana webhooks to match the security level of the GitHub integration.

💾 **Persistent Storage**

Replace in-memory event store, deduplication map, and cooldown map with a database (e.g., Redis for fast key-value, SQLite for structured traces). This enables cross-restart deduplication and historical audit trails.

🔁 **Retry Logic for Failed Agent Runs**

Add exponential backoff retries for transient failures (network errors, Claude API rate limits). Consider integrating a job queue like BullMQ for durable, distributed processing.

⚙️ **Per-Project Config for All Adapters**

Currently only GitHub reads per-repo configuration from `.github/task-ai.md`. Trello and Asana always use `defaultProjectConfig`. Adding a config-lookup mechanism for each platform would allow project-specific tech stacks and conventions.

🚀 **Mastra Workflow Builder Migration**

The workflow is currently a plain object due to complexity with Mastra's typed step API. Once Mastra improves context-passing between steps, migrating to the official `Workflow` + `createStep` pattern enables built-in step tracing, retries, and conditional branching.

🛡️ **Rate Limiting & Security Headers**

Add `express-rate-limit` middleware for global flood protection and `helmet` for security headers (CSP, HSTS, X-Frame-Options). Protect `/api/status` and `/api/logs` with at minimum a shared secret or IP allowlist.

📜 **Smart Thread Windowing**

The current 50-comment hard cap can miss early context on long issues. Implement a smart window that always includes the original issue description, the agent's first response, and the most recent N comments — ensuring the most relevant context is always present.

⏰ **Pre-Analysis Delay**

A TODO in the workflow notes that a delay before `checkDescription` would give issue authors time to finish writing. Implement via Mastra's built-in step delays or a job queue (BullMQ), so the agent waits 5–10 minutes

after the trigger before analyzing — reducing false "description too short" failures.

8 Environment Variables

Variable	Required	Description
`ANTHROPIC_API_KEY`	Required	Anthropic API key for Claude AI. Server refuses to start if absent.
`GITHUB_TOKEN`	Conditional	GitHub personal access token (PAT) with `issues:write` scope. Required to enable GitHub integration. If absent, `/webhook/github` returns 503.
`GITHUB_WEBHOOK_SECRET`	Required if GITHUB_TOKEN set	Shared secret used to verify GitHub webhook HMAC signatures. Server refuses to start if `GITHUB_TOKEN` is set but this is absent.
`GITHUB_BOT_USERNAME`	Strongly recommended	GitHub username of the bot account. Used to prevent the agent from replying to its own comments. Without this, infinite loops are possible.
`GITHUB_TRIGGER_LABEL`	Optional	Label name that triggers analysis on new issues. Defaults to `ai-review`.
`GITHUB_MENTION`	Optional	Mention string that triggers analysis in comments. Defaults to `@task-ai`.
`PORT`	Optional	HTTP port. Defaults to `3000`.
`TRELLO_API_KEY`	Phase 2	Trello API key. Not yet used (mock adapter).
`TRELLO_TOKEN`	Phase 2	Trello OAuth token. Not yet used (mock adapter).
`ASANA_TOKEN`	Phase 2	Asana personal access token. Not yet used (mock adapter).

SUMMARY

The Task Analyzer Agent is a well-structured, security-conscious Phase 1 implementation. The adapter pattern provides clean extensibility, the GitHub integration is production-ready, and the security groundwork for the most critical path (GitHub) is solid. The main areas for improvement before broader deployment are: securing the Trello/Asana webhooks, adding persistence, protecting internal APIs, and connecting RAG for richer analysis.